Offline Pipeline Libraries: Verified Facts

Verified 2026-10-18 (PyPI JSON, conda-forge API, installed and inspected where noted).

Wheel matrix

Package	Ver	aarch64 cp310	Notes
rosbags	0.11.8	pure-py	Reads MCAP and zstd. Bag-embedded msgdefs are auto-registered. Apache-2.0
open3d	0.20.0	**yes (manylinux_2_35, CPU only)**	Needs JetPack 6 (glibc 2.35)
laspy[lazrs]	2.7.0	yes	Can't write COPC
small_gicp	1.0.1	yes	`small_gicp.align(target, source, init_T_target_source, registration_type='GICP', downsampling_resolution, max_correspondence_distance, num_threads)`
kiss-icp	1.3.0	yes	Includes `kiss_icp.tools.point_cloud2.read_point_cloud`
dufomap	1.1.1	**no** (x86 only)	Build from source on Orin
py4dgeo	1.2.0	sdist (compile)	M3C2
pdal / python-pdal	conda-forge	yes	`writers.copc`
copclib	2.6.3	yes	
gsplat	1.5.3	JIT CUDA	No public Orin reports. Use the x86 RTX machine
pycolmap	4.2.1	conda-forge only	
rfdetr	1.11.2	pure-py	Apache-2.0 (Nano–Large)
sam3			SAM License (custom). Weights are gated
teaserpp	not packaged	build from source	Use Open3D FPFH+RANSAC instead

Jetson PyTorch: `pip install torch==2.8.0 torchvision==0.23.0 --index-url https://pypi.jetson-ai-lab.io/jp6/cu126`. This index is unreliable. **Do GPU-heavy work (GS training, SAM, detector training) on x86.**

Livox CustomMsg fast decode (verified)

Stride is 20 bytes. The fields are `offset_time` u32 @0, x/y/z f32 @4/8/12, reflectivity/tag/line u8 @16/17/18. Deserializing via rosbags gives Python objects at about

0.1 s per frame, so use the numpy path on the raw CDR:

```
dt = np.dtype({'names':['t','x','y','z','refl','tag','line'],'formats':
['<u4','<f4','<f4','<f4','u1','u1','u1'],
               'offsets':[0,4,8,12,16,17,18],'itemsize':20})
pts = np.frombuffer(bytes(raw)+b'\x00', dt, count=N, offset=len(raw)-(N*20-1))
```

DUFOMap

```
from dufomap import dufomap
m = dufomap(0.1, 0.2, 2, num_threads=12)  # resolution, inflate_hits_dist,
inflate_unknown
for pc_world, pose7 in scans: m.run(pc_world.astype(np.float32), pose7,
cloud_transform=False)  # pose7 = tx ty tz qw qx qy qz [verify]
m.oncePropagateCluster(if_propagate=True, if_cluster=False)
m.outputMap(all_points_world, voxel_map=False)
```

py4dgeo M3C2

```
e1, e2 = py4dgeo.Epoch(xyz1), py4dgeo.Epoch(xyz2)
m = py4dgeo.M3C2(epochs=(e1,e2), corepoints=xyz1[::50], cyl_radius=0.2, normal_radii=
[0.1,0.2,0.4])
dist, unc = m.run()   # significant where |dist| > unc["lodetection"]
```

gsplat with known poses

- **Data layout:** `data_dir/images/` + `sparse/0/{cameras,images,points3D}.txt`.
  - `images.txt` holds **world→camera** poses (OpenCV): `IMAGE_ID QW QX QY QZ TX TY TZ CAM_ID NAME`, followed by a POINTS2D line.
  - Use a PINHOLE camera with ZED rectified images.
- **Init:** `--init_type sfm` reads `points3D` + RGB. Seed it with the coloured, downsampled LiDAR map.
- **Depth loss** (`--depth_loss`) only works from point **tracks**. Write POINTS2D/tracks by projecting map points, or use av_trainer LiDAR (gsplat main / v1.6).
- **Versions:** PyPI 1.5.3 vs `main` examples; pin to the tag's examples.
- **Template:** hku-mars/GS-SDF `rosbag_to_colmap.py`.

Viewing

- **Point clouds:** PDAL `writers.copc` → static nginx with HTTP Range support → Potree `develop` `examples/copc.html`.
- **Splats:** `npx @playcanvas/splat-transform scene.ply scene.sog` (or a self-contained `.html`); Spark for embedding in three.js.

Semantics

- **Labelling:** SAM 3 via transformers (`Sam3Processor/Sam3Model`, `facebook/sam3`, text prompt), or Grounding DINO (transformers) + SAM 2.
- **Detector:** RF-DETR trained on x86 (`RFDETRMedium().train(dataset_dir=...)`) → ONNX export → engine built **on the Orin** with `/usr/src/tensorrt/bin/trtexec --onnx=... --fp16`.